

School of Computer Science & IT

Department of BCA

SOFTWARE ENGINEERING (23BCA3C01)

MODULE 1: Software Product and Process

1

Defining Software

- Software delivers the most important product of our time — information. It transforms data into information so that it can be more useful in a context; it enables you to take better decisions.
- Software is:
 - (1) **instructions** (computer programs) that when executed provide desired features, function, and performance;
 - (2) **data structures** that enable the programs to adequately manipulate information and
 - (3) **documentation** that describes the operation and use of the programs.
- Software is developed or engineered, it is not manufactured in the classical sense. Although the industry is moving toward component-based construction, most software continues to be custom-built.
- Software is a logical rather than a physical system element. So, software has one fundamental characteristic that makes it considerably different from hardware: Software doesn't "wear out."

Software Characteristics

- ❑ **1. Software is engineered.**
- ❑ **2. Software does not wear out**
- ❑ **3. Software is custom built.**

Wear vs. Deterioration

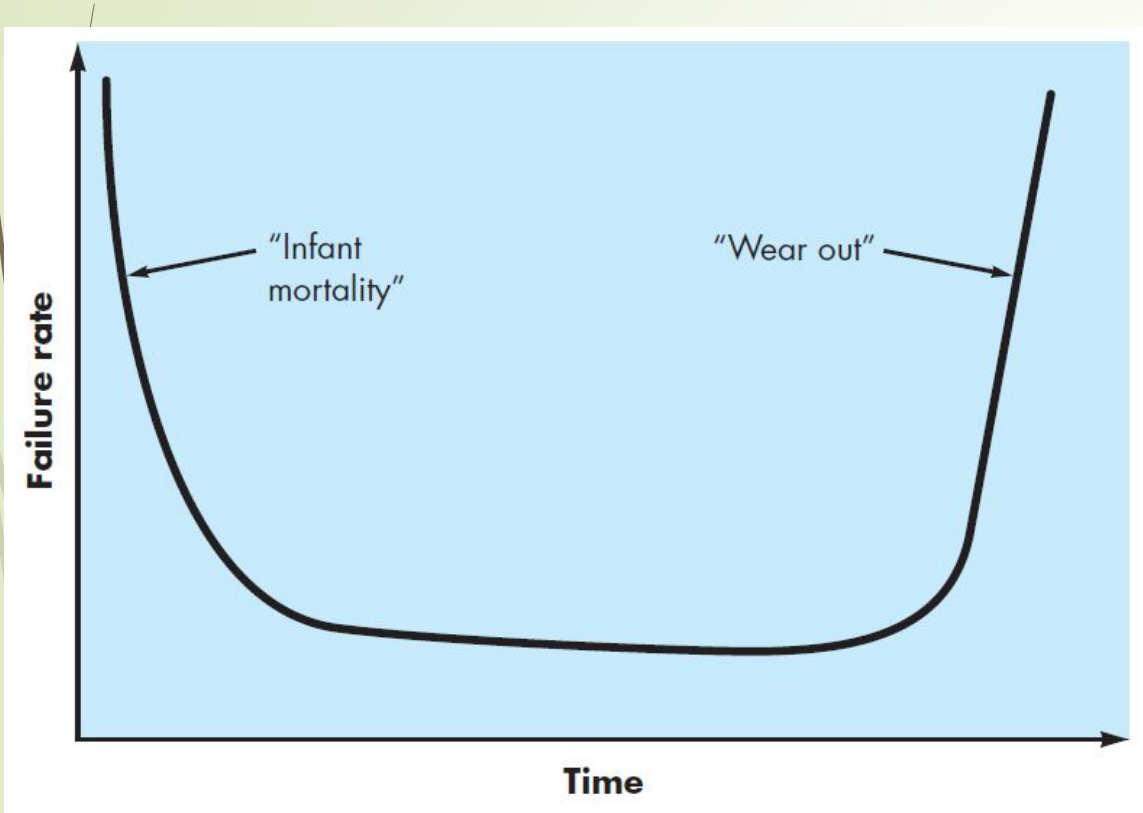


Figure: Failure curve for hardware

- Figure depicts failure rate as a function of time for hardware. The relationship, often called the “bathtub curve,” indicates that hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects); defects are corrected and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time.
- As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies.
- Stated simply, the hardware begins to wear out.
- When a hardware component wears out, it is replaced by a spare part.

Wear vs. Deterioration

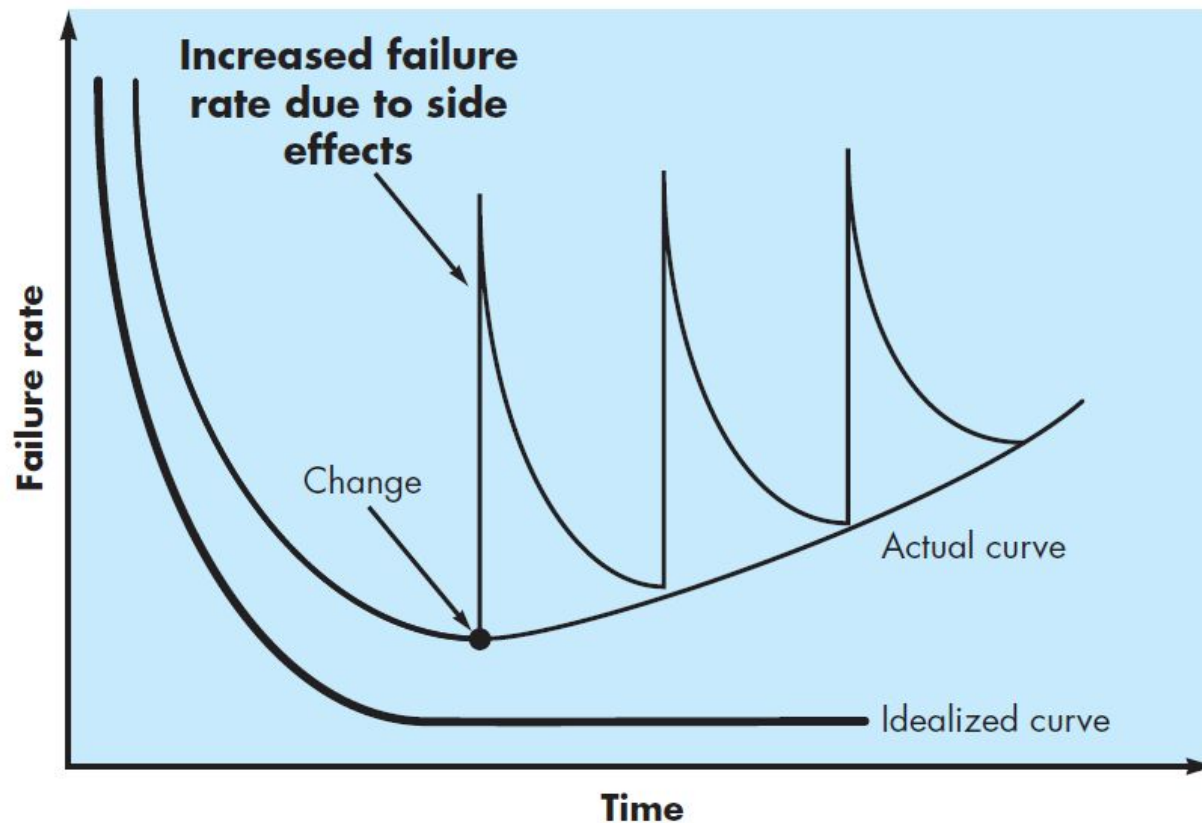


Figure: Failure curves for software

- ❑ Software is not susceptible to the environmental maladies that cause hardware to wear out.
- ❑ Therefore, theoretically, the failure rate curve for software should take the form of the “idealized curve” shown in Figure.
- ❑ Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected and the curve flattens as shown.
- ❑ The idealized curve is a gross oversimplification of actual failure models for software.
- ❑ During its life, software will undergo change. As changes are made, it is likely that errors will be introduced, causing the failure rate curve to spike as shown in the “actual curve”.
- ❑ Before the curve can return to the original steady-state failure rate, another change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise — the **software is deteriorating due to change**.
- ❑ Thus, software doesn't wear out. But it does deteriorate!

Categories of Software

- One way to categories software (but not limited):
- **System software**
 - a collection of programs written to service other programs. System software e.g., compilers, editors, and file management utilities, operating system, networking software, drivers etc.
- **Application software**
 - programs that process business data in a way that facilitates business operations or management decision making and solve a specific business need. Microsoft application software, music applications, skype, google meet, zoom etc.
- **Engineering/Scientific software**
 - a broad array of “number-crunching programs that range from astronomy to volcanology, from automotive stress analysis to orbital dynamics, and from computer-aided design to molecular biology, from genetic analysis to astronomy. MATLAB, AUTOCAD, ORCAD etc.

- **Embedded software**

- resides within a product or system and is used to implement and control features and functions for the end user and for the system itself.
- e.g., digital functions in an automobile such as fuel control, dashboard displays, and braking systems.

- **Product-line software**

- designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited marketplace (e.g., inventory control products) or address mass consumer. e.g., inventory control products) or address mass consumer.

- **Web/Mobile applications**

- this network-centric software category spans a wide array of applications and encompasses both browser-based apps and software that resides on mobile devices.

- **AI software**

- makes use of nonnumerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

Legacy Software

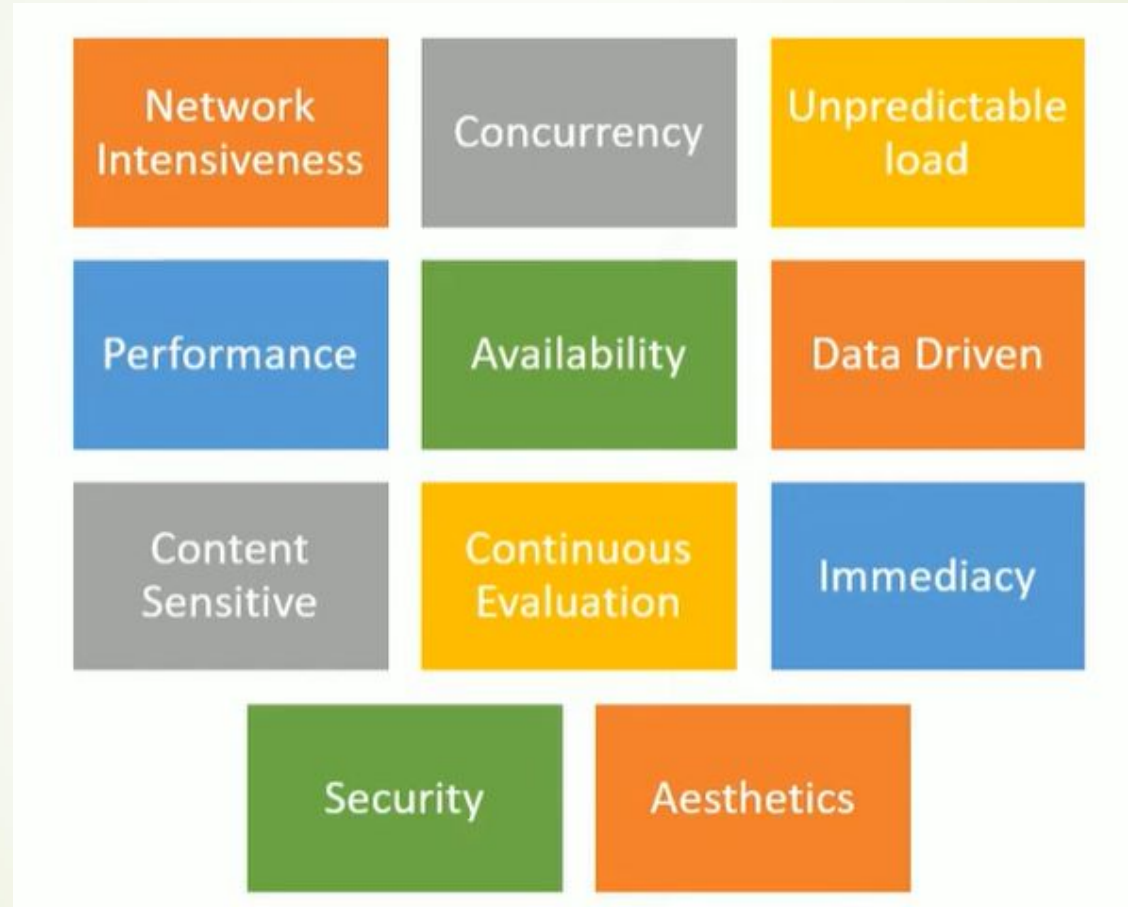
- The older software programs — often referred to as legacy software — have been the focus of continuous attention and concern.
- Legacy software can be described in the following way:
 - Legacy software systems . . . were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms. The proliferation of such systems cause headaches for large organizations who find them costly to maintain and risky to evolve.
 - Further, “many legacy systems remain supportive to core business functions and are ‘indispensable’ to the business.” Hence, legacy software is characterized by longevity and business criticality.
 - Legacy software has a characteristics – Poor Quality e.g., poor user interface, poor performance, poor documentation etc.
- As time passes, legacy systems often evolve for one or more of the following reasons:
 - software must be **adapted** to meet the needs of new computing environments or technology.
 - software must be **enhanced** to implement new business requirements.
 - software must be **extended** to make it **interoperable** with other more modern systems or databases.
 - software must be **re-architected** to make it viable within a network environment.

□ <https://www.youtube.com/watch?v=JtQ6kydHScs>

The Changing Nature of Software

- Four broad categories of software are evolving and will continue to dominate the industry.
- **WebApps**
 - Modern WebApps are much more than hypertext files with a few pictures
 - WebApps are augmented with tools like XML and Java to allow Web engineers including interactive computing capability
 - WebApps may standalone capability to end users or may be integrated with corporate databases and business applications
 - Semantic web technologies (Web 3.0) have evolved into sophisticated corporate and consumer applications that encompass semantic databases that require web linking, flexible data representation, and application programmer interfaces (API's) for access
 - The aesthetic nature of the content remains an important determinant of the quality of a WebApp.

Unique features of Webapps



Characteristics of WebApps

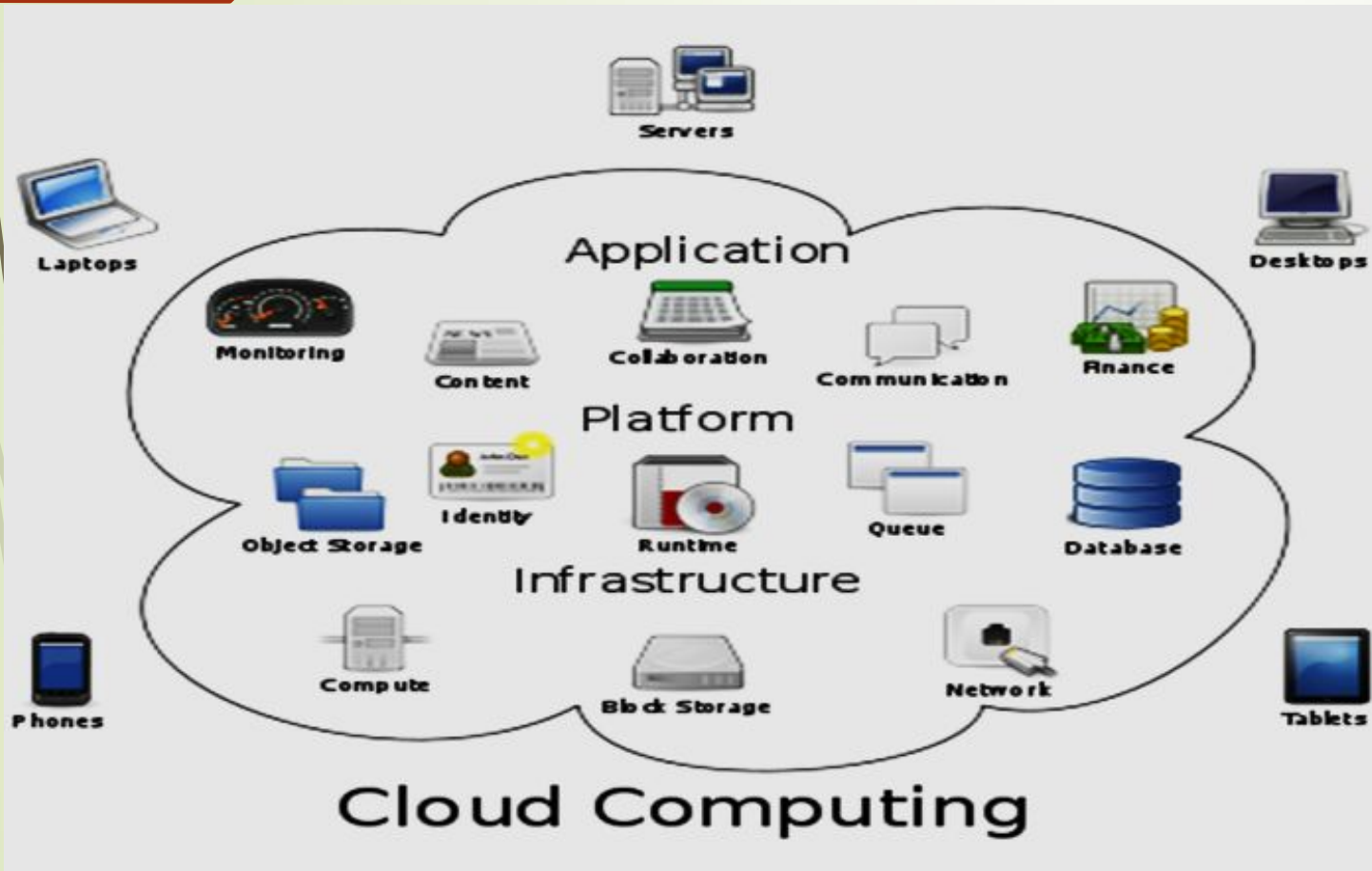
- **Data driven.** The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end-user.
- **Content sensitive.** The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.
- **Continuous evolution.** Unlike conventional application software that evolves over a series of planned, chronologically-spaced releases, Web applications evolve continuously.
- **Immediacy.** Although *immediacy*—the compelling need to get software to market quickly—is a characteristic of many application domains, WebApps often exhibit a time to market that can be a matter of a few days or weeks.
- **Security.** Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end-users who may access the application.
- **Aesthetics.** An undeniable part of the appeal of a WebApp is its look and feel.

WHAT IS WEB APP



❑ Mobile Apps

- Reside on mobile platforms such as cell phones or tablets
- Contain user interfaces that take both device characteristics and location attributes
- Often provide access to a combination of web-based resources and local device processing and storage capabilities
- Provide persistent storage capabilities within the platform
- *A mobile web application* allows a mobile device to access to web-based content using a browser designed to accommodate the strengths and weaknesses of the mobile platform
- *A mobile app* can gain direct access to the hardware found on the device to provide local processing and storage capabilities
- As time passes these differences will become blurred



Cloud Computing

- Cloud computing provides distributed data storage and processing resources to networked computing devices
- Computing resources reside outside the cloud and have access to a variety of resources inside the cloud
- Cloud computing requires developing an architecture containing both frontend and backend services
- Frontend services include the client devices and application software to allow access
- Backend services include servers, data storage, and server-resident applications
- Cloud architectures can be segmented to restrict access to private data

❑ Product Line Software

- *Product line software* is a set of software-intensive systems that share a common set of features and satisfy the needs of a particular market
- These software products are developed using the same application and data architectures using a common core of reusable software components
- A software product line shares a set of assets that include *requirements, architecture, design patterns, reusable components, test cases*, and other work products
- A software product line allow in the development of many products that are engineered by capitalizing on the commonality among all products within the product line.

THANK YOU

Any questions...?

